

```
#define SIMULA_DEFAULT_FILE ".sfsf"
typedef unsigned int uint4_t;

typedef struct sfs_super_block
{
    uint4_t type; /* Magic number to identify the file system */
    uint4_t block_size; /* Unit of allocation */
    uint4_t partition_size; /* in blocks */
    uint4_t entry_size; /* in bytes */
    uint4_t entry_table_size; /* in blocks */
    uint4_t entry_table_block_start; /* in blocks */
    uint4_t entry_count; /* Total entries in the file system */
    uint4_t data_block_start; /* in blocks */
    uint4_t reserved[SIMULA_FS_BLOCK_SIZE / 4 - 8];
} sfs_super_block_t; /* Making it of SIMULA_FS_BLOCK_SIZE */

typedef struct sfs_file_entry
{
    char name[16];
    uint4_t size; /* in bytes */
    uint4_t timestamp; /* Seconds since Epoch */
    uint4_t perms; /* Permissions for user */
    uint4_t blocks[SIMULA_FS_DATA_BLOCK_CNT];
} sfs_file_entry_t;

#endif
```

Note that Shweta has put some redundant fields in the `sfs_super_block_t` rather than computing them every time. Practically, that is not space inefficient, as in any case, there is a lot of empty reserved space available in the super block, which is expected to be the complete first block of a partition. For example, `entry_count` is a redundant field as it is the same as the `entry_table`'s size (in bytes) divided by the entry size, with both being part of the super block structure. Moreover, the `data_block_start` number could also be computed by how many blocks have been used before it, but this is not preferred.

Also, note the hard coded assumptions made about the block size of 512 bytes, and some random magic number for the simula file system. This magic number is used to verify that we are dealing with the right file system, when operating on it.

Coming to the file entry, it contains the following for every file:

- Name of up to 15 characters (1 byte for '\0')
- Size in bytes
- Time stamp
- Permissions (just one set, for the user)
- Array of block numbers for up to 9 data blocks. Why 9? To make the complete entry of `SIMULA_FS_ENTRY_SIZE` (64).

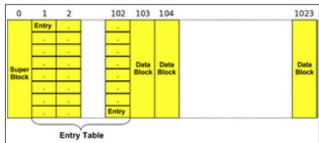


Figure 1: Simula file system on 512 KB of partition (.sfsf)

With the file system design ready, the make file system (mkfs) or the more commonly known format application is implemented next, as *format_sfs.c*:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

#include "sfs_ds.h"

#define SFS_ENTRY_RATIO 0.10 /* 10% of all blocks */
#define SFS_ENTRY_TABLE_BLOCK_START 1

sfs_super_block_t sb =
{
    .type = SIMULA_FS_TYPE,
    .block_size = SIMULA_FS_BLOCK_SIZE,
    .entry_size = SIMULA_FS_ENTRY_SIZE,
    .entry_table_block_start = SFS_ENTRY_TABLE_BLOCK_START
};

sfs_file_entry_t fe; /* All 0's */

void write_super_block(int sfs_handle, sfs_super_block_t *sb)
{
    write(sfs_handle, sb, sizeof(sfs_super_block_t));
}

void clear_file_entries(int sfs_handle, sfs_super_block_t *sb)
{
    int i;

    for (i = 0; i < sb->entry_count; i++)
    {
        write(sfs_handle, &fe, sizeof(fe));
    }
}

void mark_data_blocks(int sfs_handle, sfs_super_block_t *sb)
{
    char c = 0;
```